Strategic Planning and Data Exploration for SwiftRoute Logistics Pvt. Ltd.: Inventory Management and Last-Mile Delivery Optimisation

1. Logistics Scenario:

SwiftRoute is a mid-sized Logistics and e-commerce delivery company that operates a warehouse and manages inventory, order fulfilment, transportation and last-mile delivery across an urban and peri-urban service area. The company handles online customer orders, consumer products, different product categories, small- and medium-sized packages, and orders from customers in urban and peri-urban areas. When a customer places an order, the company records information such as; Order date/time, product, quantity, customer/location, delivery time, delivery status. The warehouse receives products from suppliers, stores products, tracks available stocks, updates inventory when orders are received, replenishes products when stocks are low, dispatches products for customer orders. After an order is packed orders are grouped based on delivery locations, available vehicles are assigned, routes are planned, vehicle capacity and delivery schedules need to be considered, the riders deliver the orders to customers. The company faces problems such as late deliveries, inefficient routes, longer delivery times, failed/repeated delivery attempts, stock-outs, slow-moving products, high transportation costs, unnecessary travel distance, underutilized vehicle capacity, difficulty predicting demand.

2. Business Problem Statement:

The SwiftRoute Logistics Pvt. Ltd. faces delivery delays and inventory imbalances affecting operational efficiency. The company wants to analyze historical order, inventory and delivery data to identify the factors contributing to these problems and support better inventory planning, delivery scheduling and transportation resource allocation.

3. Project Objectives:

1. **Analyze demand patterns**

Study past order data and understand demand patterns across products, locations and time periods.

2. **Analyze inventory performance**
   Evaluate inventory performance to identify stock-out risks, slow-moving products and potential inventory imbalances.

3. **Measure delivery performance**
   Analyze delivery performance and identify patterns associated with delays and unsuccessful delivery attempts.

4. **Analyze transportation efficiency**
   Study delivery route operations, evaluate route efficiency, vehicle utilization and delivery-related transportation costs.

5. **Forecast future demand**
   Develop a demand forecasting approach to estimate future product demand and support inventory planning.

6. **Segment customers or delivery zones**
   Group customers or delivery zones based on order volume, frequency, distance and other relevant operational characteristics to identify distinct logistics patterns.

7. **Develop a route optimization approach**
   Find more efficient delivery routes by considering vehicle capacity and delivery requirements.

8. **Evaluate potential business impact**
   Measure how data analysis can improve delivery, inventory, transportation, and resource use.

4. Key Performance Indicators

1. **On-Time Delivery Rate**- Measures how many orders are delivered on time.
2. **Stock-Out Rate**- Measures how often products are unavailable when needed.

3. **Average Delivery Time**- Measures the average time taken to deliver an order.
4. **Delivery Cost per Order**- Measures the average transportation cost for each order.
5. **Vehicle Utilization**- Measures how efficiently the available vehicle capacity is being used.
6. **Inventory Turnover**- Measures how quickly inventory is sold or used.
7. **Order Cycle Time**- Measures total time taken from receiving an order until order is delivered to the customer.

5. Literature and Data Research

SwiftRoute is a hypothetical logistics and e-commerce company used in this project serving urban and peri-urban markets, while publicly available datasets and research will be used to understand and demonstrate the proposed analytical approach. Research shows that demand forecasting, last-mile optimization, logistic-KPIs and operational efficiency can help businesses plan inventory more effectively.

5.1 Demand Forecasting and Inventory Management:

Demand forecasting is a process of predicting customers' future demand using historical data other relevant information. It helps businesses handle with inventory demand planning, transportation capacity planning, and warehouse resourcing and reduce the risk of having too much or too little stock. For SwiftRoute, demand forecasting can be used to estimate future product demand and support better inventory planning.

5.2 Last- Mile Delivery and Route Optimization:

Route optimization is a process of determining the most efficient delivery routes for deliveries or pickups taking into account factors like distance, time, vehicle capacity, and other constraints with the goal of reducing costs and maximizing efficiency. For SwiftRoute, Route Optimization can help in planning better delivery routes, timely deliveries, proper vehicle utilization reducing costs and improve operational efficiency.

5.3 Logistics KPIs and Operational Efficiency:

Logistics KPIs are the quantifiable metrics used to measure, track, and evaluate efficiency, costs, and quality of supply chain and fulfilment operations. For SwiftRoute, KPIs can help the company identify delivery problems, monitor transportation performance, measure transportation efficiency, compare performance over time, and identify areas that need improvement. The selected KPIs for this project are; On-Time Delivery, Stock-out Rate, Delivery Cost per Order, Average Delivery Time, Vehicle Utilization, Inventory Turnover, and Order Cycle Time.

5.4 Public Datasets for Logistics/E-commerce Analysis:

SwiftRoute is a hypothetical logistics and e-commerce company serving urban and peri-urban markets. The public dataset is only being used as a research data source to demonstrate analytical approach, and not as SwiftRoute's actual company data.

Dataset Name: UCI Online Retail Dataset

Source: UCI Machine Learning Repository

What it contains: Transaction-level online retail information such as products, quantities, dates, prices, and customers.

How it helps SwiftRoute: UCI Online Retail Dataset demonstrate the proposed analytical approach demand analysis, product analysis, customer segmentation, and time-based analysis.

Limitation: The dataset does not contain detailed delivery, vehicle routes, transportation costs or inventory information, so those parts of the SwiftRoute project would require additional data.

6. Data Science Methodologies

6.1 EDA (Exploratory Data Analysis):

EDA is an investigative process where we summarize, visualize, and study a dataset to find missing values, hidden patterns, spot anomalies, and check data quality before building any formal models. For SwiftRoute, patterns EDA can identify patterns such as which products have the highest demand, which locations have more orders, which products are slow-moving, how often stock-outs occur, when are the orders highest, are there any unusual or missing values, which factors appear related to delivery delays, which routes involve longer distances,

areas or routes have higher delivery costs. Problems EDA can detect are missing data, duplicate records, outliers, stock-out situations, slow-moving products, delivery delays, inefficient routes, and underutilised vehicles.

6.2 Demand Forecasting:

Demand forecasting is a process of predicting future sales using statistical methods and historical data. Historical data such as data quantity, product history, date/time records, location, seasonality/trends performance. For SwiftRoute, demand forecasting would help in optimizing stock levels, reducing holding costs, managing storage capacity, improving slotting efficiency. Forecast techniques to use in this project are:

Moving Average – simple baseline forecasting.
Linear Regression – identify trends and estimate future demand.
Time-Series Forecasting – analyze demand patterns over time.

6.3 Regression:

Regression is a statistical method used to find the relationship between a target variable and one or more input variables so we can predict future values. For SwiftRoute, for example:

Target: Delivery Time
Inputs: Distance, order quantity, vehicle type, number of stops, delivery zone, time of the day.

The regression model would learn relationships between these variables and then estimate the delivery time for a new order. This could help SwiftRoute predict delivery time, identify potentially slow deliveries, improve scheduling/resources, and improve overall delivery performance.

6.4 Clustering:

Clustering is an unsupervised machine learning method used to group similar observations together based on their characteristics. SwiftRoute could group customers or delivery zones based on characteristics like order volume, order frequency, average delivery distance, average order quantity, delivery cost, delivery frequency. These groups could

help SwiftRoute identify high- and low-demand areas, understand different customer patterns, plan delivery resources, improve delivery scheduling, and design different operational strategies for different zones.

6.5 Route Optimization:

Route optimization means finding a more efficient sequence of delivery routes/stops while considering operational constraints. The goals of route optimization for SwiftRoute could be reduce total travel distance, reduce delivery time, reduce transportation costs, improve vehicle utilization, meet delivery requirements, handle vehicle-capacity constraints. The optimization process would also consider constraints including inefficient routes, unnecessary travel distance, high transportation costs, and underutilized vehicle capacity. A good route optimization approach could help SwiftRoute plan better routes, reduce travel distance and transportation cost, improve vehicle utilization, and increase delivery efficiency.

7. Strategic Roadmap

7.1 Business Understanding:

The first stage of the project is to understand the major business problems faced by SwiftRoute Logistics Pvt. Ltd. The company needs to address delivery delays, inventory imbalances, inefficient routes, high transportation costs, underutilized vehicle capacity, and difficulty in predicting demand. The analysis will support better inventory planning, delivery scheduling, transportation resource allocation, and overall operational efficiency. The overall goal is to use data-driven analysis to identify operational problems, improve logistics performance, and support better business decision-making.

7.2 Data Collection:

The next stage is to collect data required for analysing SwiftRoute's logistics operations. The required data would include order data such as order ID, order date and time, product, quantity, customer, and delivery location. Inventory data would include available stock, stock issued, stock-out occurrences, and replenishment information. Delivery, vehicle, and route data would include dispatch time, delivery time, delivery status, delivery attempts, vehicle type, vehicle capacity, route,

distance travelled, number of stops, and transportation cost. Customer and location data would help analyse order frequency, delivery distance, and demand patterns across different delivery zones. These data could be collected from different operational sources such as order management, warehouse, inventory, and delivery systems. For this project, the UCI Online Retail Dataset will be used to demonstrate parts of the proposed analytical approach because it does not contain detailed delivery, vehicle, route, transportation cost, or inventory information.

7.3 Data Quality Assessment:

After collecting the required data, the next stage is to assess its quality before performing detailed analysis. The data will be checked for missing values, duplicate records, incorrect data types, invalid values, outliers, and inconsistent formats. For example, order quantities, dates, delivery times, and transportation costs should contain valid and appropriate values. Location and category information will also be checked for inconsistent entries that may represent the same value. The purpose of this stage is to identify data-quality problems and determine which issues need to be addressed before data cleaning, EDA, and data modelling.

7.4 Data Cleaning and Preparation:

After assessing data quality, the identified issues will be addressed to prepare the dataset for analysis. Missing values will be handled based on the type and importance of data, while duplicate records will be identified and removed where appropriate. Incorrect data types and inconsistent formats, such as dates, quantities, costs, product categories, and location names, will be standardized. Invalid values and unusual observations will be investigated and corrected or handled appropriately rather than automatically removing them. After cleaning, the required datasets will be prepared in a consistent and analysis-ready format for EDA, KPI calculation, demand forecasting, regression, clustering, and route optimization.

7.5 Feature Engineering:

Feature engineering means creating useful variables/features from existing data so that they can be used effectively for analysis and modelling. It is a very important stage after data cleaning; relevant features will be created from the available variables to support analysis and modelling. For example, delivery duration can be calculated using dispatch time and delivery time, while order cycle time can be calculated using order time and delivery time. Vehicle utilization can be calculated comparing actual load with the available vehicle capacity, and delivery cost per order can be derived from total transportation cost and the number of orders. Additional demand-related features such as daily, weekly, or monthly order quantities and order frequency by delivery zones can also be created. These engineered features will support KPI calculation, EDA, demand forecasting, regression, clustering, and route optimization.

7.6 EDA (Exploratory Data Analysis):

After data cleaning and feature engineering, exploratory Data analysis (EDA) will be performed to understand the main patterns and relationships in the SwiftRoute logistics data. The analysis will examine order demand, inventory performance, delivery performance, transportation efficiency, and customer or delivery-zone patterns.

The analysis will include summary statistics and visualizations to identify high-demand products, peak order periods, high-and low-demand delivery zones, slow-moving products, stock-out patterns, delivery delays, delivery distances, transportation costs, and vehicle utilization. The relationships between variables such as distance and delivery time, order quantity and delivery time, and delivery zone and delivery performance will also be explored.

Different charts can be used depending on the variable being analyzed, such as bar charts for product or zone comparisons, line charts for demand trends over time, histograms for distributions, and scatter plots for relationships between numerical variables. EDA will also help identify unusual observations or patterns that may require further investigation before applying predictive models.

The findings from EDA will be used to select relevant variables for demand forecasting, regression, clustering, and route optimization and to support the development of data-driven business recommendations.

7.7 Predictive Modelling and Analysis:

After EDA, predictive modelling will be performed to estimate future demand and understand factors that affect logistics performance. The modelling stage will depend on the variables identified during EDA and will depend on the availability and quality of the required data.

For demand forecasting, historical order quantities will be used to estimate future product demand. Moving Average can be used as a simple baseline, while regression or time-series forecasting methods can be applied to identify trends and predict future demand.

For delivery performance, regression can be used to estimate delivery time using variables such as delivery distance, order quantity, vehicle type, number of stops, delivery zone, and time of day. The model results can help identify orders or delivery conditions that may result in longer delivery times.

Clustering can be used to group customers or delivery zones with similar characteristics, such as order volume, order frequency, delivery distance, and delivery cost. These groups can help SwiftRoute understand different demand and delivery patterns and support better allocation of delivery resources.

The performance of the predictive models will be evaluated using suitable measures and the results will be compared with actual observations where possible. The findings will then be used to support inventory planning, delivery scheduling, resource allocation, and other operational decisions.

7.8 Route Optimization:

After analysing delivery performance and identifying relevant patterns, a route optimization approach will be developed to improve the efficiency of delivery operations. The objective will be to find suitable delivery sequences while considering factors such as delivery locations, distance, vehicle capacity, number of stops, and delivery requirements.

The analysis will first identify inefficient routes, unnecessary travel distance, high transportation costs, and low vehicle utilization. Based on these findings, delivery stops can be grouped and arranged into more efficient routes. Vehicle capacity and delivery requirements will be considered so that the proposed routes remain practical for the available vehicles.

The performance of the optimized routes can be compared with existing routes using measures such as total travel distance, delivery time, transportation cost, and vehicle utilization. This comparison will help determine whether the proposed approach can improve logistics efficiency.

The route optimization results will support better delivery scheduling, reduce unnecessary travel, improve vehicle utilization, control transportation costs, and contribute to better on-time delivery performance.

7.9 KPI Evaluation and Performance Measurement:

After completing the analytical and optimization stages, the key performance indicators (KPIs) will be evaluated to measure logistics performance and assess the potential impact of the proposed improvements. The selected KPIs will be calculated using the prepared data and compared across different products, delivery zones, time periods, and operational conditions where applicable.

The analysis will evaluate On-Time Delivery Rate, Stock-Out rate, Average Delivery Time, Delivery Cost per Order, Vehicle Utilization, Inventory Turnover, and Order Cycle Time. These KPIs will help measure delivery performance, inventory efficiency, transportation efficiency, and overall operational performance.

Where sufficient historical data is available, KPI performance before and after implementing proposed improvements can be compared. For example, optimized routes can be evaluated based on changes in travel distance, delivery time, transportation cost, and vehicle utilization. Similarly, demand forecasting can be evaluated based on its ability to support better inventory planning and reduce stock-out risks.

The KPI results will be used to identify areas of improvement and determine whether the proposed data-driven approaches are contributing to better logistics performance. The findings will provide measurable evidence to support business recommendations and operational decision-making.

7.10 Business recommendations and Decision-Making:

The findings from the EDA, predictive modelling, clustering, route optimization, and KPI evaluation will be converted into practical business recommendations for SwiftRoute. The recommendations will focus on improving inventory planning, delivery performance, transportation efficiency, and resource allocation.

If demand analysis and forecasting identify products with consistently high or increasing demand, SwiftRoute can consider maintaining appropriate stock levels and planning replenishment accordingly. Slow-moving products can be reviewed to avoid unnecessary inventory accumulation and storage use.

If the analysis identifies delivery zones or routes with frequent delays, long delivery times, or high transportation costs, the company can review delivery schedules and route plans for those areas. The route optimization results can be used to reduce unnecessary travel and improve vehicle utilization.

Customer or delivery-zone clusters can also support different operational strategies based on demand volume, delivery frequency, distance, and cost characteristics. Regression results can help identify conditions associated with longer delivery times and support better delivery planning.

The selected KPIs will be used to monitor whether these actions are improving business performance. Overall, the recommendations will help SwiftRoute make data-driven decisions, improve operational efficiency, control logistics costs, and provide better delivery service.

8. Python Implementation / Code Illustrations

8.1 Importing Libraries:

Python provides several libraries that can be used to perform data analysis, visualization, and machine learning for the SwiftRoute Logistics project. The following libraries would be useful for the analysis:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

The pandas library can be used for loading, cleaning, transforming, and analyzing tabular data. Numpy provides numerical operations, while matplotlib and seaborn can be used to create visualizations for exploratory data analysis.

The scikit-learn library can be used for predictive modelling and clustering. For example, LinearRegression can be used to demonstrate regression analysis and KMeans can be used for customer or delivery-zone clustering.

The exact libraries and model used in the final SwiftRoute implementation would depend on the available operational data and the requirements identified during the data exploration stage.

8.2 Loading the UCI Online Retail Dataset:

The UCI Online Retail Dataset will be used to demonstrate the initial stages of the proposed data analysis process. The dataset contains transaction-level information such as products, quantities, dates, prices, and customers. It can therefore be used to demonstrate demand analysis, product analysis, customer analysis, and time-based analysis.

The dataset can be loaded into Python using pandas library:

```
# Load the dataset
```

```
df = pd.read_excel("Online Retail.xlsx")

# Display the first five records
print(df.head())
```

The read_excel() function is used because the dataset is provided in Excel format. The head() function displays the first five rows, allowing the analyst to confirm that the dataset has been loaded correctly.

The dataset can then be inspected to understand its structure:

```
# Check number of rows and columns
print(df.shape)

# Display column names
print(df.columns)

# Check data types
print(df.dtypes)

# Display basic information about the dataset
print(df.info())
```

These checks provide an initial understanding of this dataset, including the number of observations, available, variables, and data types. This information will be useful for the subsequent data quality assessment and cleaning stages.

It is important to note that this dataset is being used only to demonstrate the analytical approach. It does not contain SwiftRoute-specific delivery, vehicle, route, transportation cost, or inventory information. Those analyses would require additional operational data from the company.

8.3 Initial Data Inspection:

After loading the dataset, the next step is to inspect the data before performing cleaning and analysis. Initial inspection helps understand the

structure of the dataset and identify possible data-quality issues such as missing values, duplicate records, and unusual values.

The following Python commands can be used:

```
# Display the first five rows
print(df.head())

# Display the last five rows
print(df.tail())

# Check the number of rows and columns
print("Shape of dataset:", df.shape)

# Check column names
print("Columns:", df.columns.tolist())

# Check data types and non-null values
print(df.info())

# Check missing values
print("Missing values:")
print(df.isnull().sum())

# Check duplicate records
print("Number of duplicate rows:", df.duplicated().sum())

# Generate descriptive statistics
print(df.describe())
```

The head() and tail() functions help examine sample records from the beginning and end of the dataset. The shape attribute provides the number of rows and columns, while columns display the available variables.

The info() function provides information about the data types and non-null values in each column. The isnull().sum() command helps identify missing values, while duplicated().sum() identifies duplicate records.

The describe() function provides descriptive statistics for numerical variables, such as count, mean, standard deviation, minimum, and maximum values.

This initial inspection will help determine which data-quality issues need to be addressed during the data cleaning stage. It also provides an understanding of the variables available for subsequent exploratory analysis.

8.4 Data Cleaning and Preparation:

After inspecting the data, the next step is to clean and prepare the data for analysis. The UCI Online Retail Dataset may contain missing values, duplicate records, incorrect data types, cancelled transactions, and unusual values. These issues need to be handled carefully before performing EDA or modelling.

The following steps can be used for data cleaning:

```
# Check missing values
print(df.isnull().sum())

# Check duplicate records
print("Duplicates:", df.duplicated().sum())

# Remove duplicate records
df = df.drop_duplicates()

# Convert InvoiceDate to datetime format
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

# Check negative quantities
print("Negative quantities:", (df["Quantity"] < 0).sum())

# Check negative or zero prices
print("Invalid prices:", (df["UnitPrice"] <= 0).sum())
```

The missing values should be examined before deciding whether to remove or replace them. For example, customer information may be

missing for some transactions, while other fields may be required for a particular analysis.

Duplicate records can be removed where they represent repeated entries rather than the genuine transactions. The InvoiceDate column is converted into proper datetime format so that the date can be used for time-based analysis.

Negative quantities require additional investigation because they may represent cancelled or returned transactions rather than normal sales. Therefore, they should not automatically be treated as errors.

For analyses focused specifically on completed sales, cancelled transactions can be excluded after confirming how cancellation records are represented in the dataset.

```
# Identify cancelled transactions
cancelled = df["InvoiceNo"].astype(str).str.startswith("C")

print("Cancelled transactions:", cancelled.sum())

# Create a sales-only dataset
sales_df = df[~cancelled].copy()

# Remove transactions with invalid unit prices
sales_df = sales_df[sales_df["UnitPrice"] > 0]

# Remove transactions with non-positive quantities
sales_df = sales_df[sales_df["Quantity"] > 0]

print("Cleaned dataset shape:", sales_df.shape)
```

After these steps, the resulting sales_df dataset can be used for further analysis. The cleaning process should always be documented because removing or modifying records can affect the final analytical results.

8.5 Feature Engineering:

After cleaning the dataset, the next step is feature engineering. Feature engineering means creating new variables from existing data so that the dataset can provide more meaningful information for analysis and modelling.

For the UCI Online Retail Dataset, features such as total sales value, year, month, day, day of week, and hour can be created from the available transaction data.

```
# Create a copy of the cleaned sales dataset
sales_df = sales_df.copy()

# Calculate total sales value
sales_df["TotalSales"] = sales_df["Quantity"] * sales_df["UnitPrice"]

# Extract date and time features
sales_df["Year"] = sales_df["InvoiceDate"].dt.year
sales_df["Month"] = sales_df["InvoiceDate"].dt.month
sales_df["Day"] = sales_df["InvoiceDate"].dt.day
sales_df["DayOfWeek"] = sales_df["InvoiceDate"].dt.dayofweek
sales_df["Hour"] = sales_df["InvoiceDate"].dt.hour

# Display the new features
print(sales_df[
    ["InvoiceDate", "Quantity", "UnitPrice", "TotalSales",
     "Year", "Month", "Day", "DayOfWeek", "Hour"]
].head())
```

The TotalSales feature represents the sales value of each transaction and is calculated by multiplying the quantity purchased by the unit price.

The date and time features allow the analyst to study sales patterns across different years, months, days, days of the week, and hours. These features can be particularly useful for identifying peak demand periods and understanding changes in customer purchasing behaviour.

Additional aggregated features can be also created for product and customer analysis.

```
# Product-level sales analysis
product_summary = sales_df.groupby("Description").agg(
    Total_Quantity=("Quantity", "sum"),
    Total_Sales=("TotalSales", "sum"),
    Number_of_Transactions=("InvoiceNo", "nunique")
).reset_index()

# Customer-level analysis
customer_summary = sales_df.groupby("CustomerID").agg(
    Total_Quantity=("Quantity", "sum"),
    Total_Sales=("TotalSales", "sum"),
    Number_of_Orders=("InvoiceNo", "nunique")
).reset_index()

print(product_summary.head())
print(customer_summary.head())
```

These features will support the next stages of the project, including demand analysis, product analysis, customer segmentation, time-based analysis, and exploratory data analysis.

For the actual SwiftRoute implementation, additional operational features would be created from company data, such as delivery duration, order cycle time, vehicle utilization, delivery cost per order, delivery distance, and delivery zone characteristics. These features are part of the proposed SwiftRoute analytical framework but cannot be calculated from the UCI Online Retail Dataset because the required operational fields are not available.

8.6 Exploratory Data Analysis (EDA):

After cleaning the dataset and creating useful features, exploratory data analysis (EDA) will be performed to understand the main patterns, trends, and relationships in the data. EDA helps identify high- demand products, sales trends, peak purchasing periods, and customer behaviour.

8.6.1 Basic Summary Statistics:

The first step is to calculate basic summary statistics to numerical variables.

```
# Summary statistics
print(sales_df[["Quantity", "UnitPrice", "TotalSales"]].describe())
```

This provides information such as the average, minimum, maximum, and standard deviation of quantity, unit price, and total sales.

8.6.2 Total Sales by Month:

Monthly sales can be analyzed to identify changes in demand over time.

```
# Calculate monthly sales
monthly_sales = sales_df.groupby(
    ["Year", "Month"]
)["TotalSales"].sum().reset_index()

print(monthly_sales)
```

A line chart chart can then be used to visualize the monthly trend.

```
plt.figure(figsize=(10, 5))

plt.plot(
    monthly_sales.index,
    monthly_sales["TotalSales"],
    marker="o"
)

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

The analysis can help identify periods of increasing or decreasing demand.

8.6.3 Top Products by Quantity:

The products with the highest quantities sold can be identified using the product-level summary created during feature engineering.

```
# Top 10 products by quantity sold
top_products = product_summary.sort_values(
    "Total_Quantity",
    ascending=False
).head(10)

print(top_products)
```

A bar chart can be used to visualize the results.

```
plt.figure(figsize=(10, 6))

plt.barh(
    top_products["Description"],
    top_products["Total_Quantity"]
)

plt.title("Top 10 Products by Quantity Sold")
plt.xlabel("Quantity Sold")
plt.ylabel("Product")
plt.gca().invert_yaxis()

plt.tight_layout()
plt.show()
```

This helps identify products with high demand and can support inventory planning in the proposed SwiftRoute scenario.

8.6.4 Sales Distribution:

The distribution of transaction sales vales can also be examined.

```
plt.figure(figsize=(8, 5))

plt.hist(
    sales_df["TotalSales"],
    bins=50
```

```
)

plt.title("Distribution of Transaction Sales")
plt.xlabel("Transaction Sales Value")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

The histogram helps understand how transaction values are distributed and whether there unusually high or low transaction values.

8.6.5 Relationship Between Quantity and Sales:

A scatter plot can be used to examine the relationship between quantity purchased and total sales.

```
plt.figure(figsize=(8, 5))

plt.scatter(
    sales_df["Quantity"],
    sales_df["TotalSales"],
    alpha=0.3
)

plt.title("Quantity vs Total Sales")
plt.xlabel("Quantity")
plt.ylabel("Total Sales")

plt.tight_layout()
plt.show()
```

This analysis can help identify whether larger quantities are generally associated with higher transaction values and whether unusual observations are present.

This analysis can help identify whether larger quantities are generally associated with higher transaction values and whether unusual observations are present.

The findings from EDA will be used to guide subsequent demand analysis, forecasting, customer segmentation, and other analytical activities. In the actual SwiftRoute implementation, EDA would additionally examine inventory levels, delivery times, delivery distances, transportation costs, vehicle utilization, and delivery-zone performance using the company's operational data.

8.7 Demand Analysis:

After performing EDA, the next step is analyze demand patterns in more detail. Demand Analysis helps understand which products are purchased more frequently, how demand changes over time, and which periods have higher or lower order activity. For SwiftRoute, this analysis can support inventory planning, warehouse resource allocation, and delivery capacity planning.

8.7.1 Daily Demand Analysis:

Daily demand can be calculated by grouping the cleaned transaction data by date and adding the quantities sold.

```
# Create a date column
sales_df["Date"] = sales_df["InvoiceDate"].dt.date

# Calculate daily demand
daily_demand = sales_df.groupby("Date")["Quantity"].sum().reset_index()

print(daily_demand.head())
```

This analysis provides the total quantity sold on each day. It can help identify days with unusually high or low demand.

A line chart can be used to visualize the daily demand pattern:

```
plt.figure(figsize=(12, 5))

plt.plot(
   daily_demand["Date"],
   daily_demand["Quantity"]
```

```
)

plt.title("Daily Demand Trend")
plt.xlabel("Date")
plt.ylabel("Quantity Sold")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

The resulting trend can help identify changes in demand over time and periods with higher or lower activity.

8.7.2 Demand by Day of the Week:

Demand can also be compared across different days of the week.

```
weekly_demand = sales_df.groupby(
   "DayOfWeek"
)["Quantity"].sum().reset_index()

print(weekly_demand)
```

The DayOfWeek feature represents the day of the week, where Monday is represented by 0 and Sunday by 6.

A bar chart can be used to compare demand across the days:

```
plt.figure(figsize=(8, 5))

plt.bar(
   weekly_demand["DayOfWeek"],
   weekly_demand["Quantity"]
)

plt.title("Demand by Day of the Week")
plt.xlabel("Day of Week")
plt.ylabel("Quantity Sold")
plt.tight_layout()
plt.show()
```

This analysis can help identify which days generally have higher demand. For SwiftRoute, such information could support warehouse staffing, inventory preparation, and delivery resource planning.

8.7.3 Product Demand Analysis:

Product-level demand can be analyzed using the product_summary dataset created during feature engineering.

```
product_demand = product_summary.sort_values(
   "Total_Quantity",
   ascending=False
)

print(product_demand.head(10))
```

This identifies products with the highest total quantities sold.
High- demand products may require closer inventory monitoring and appropriate replenishment planning, while products with consistently low demand may require further review.

8.7.4 Demand by Month:

Monthly demand can also be calculated to identify broader changes in demand.

```
monthly_demand = sales_df.groupby(
   ["Year", "Month"]
)["Quantity"].sum().reset_index()

print(monthly_demand)
```

The results can be used to compare demand across different months and identify periods of increasing or decreasing.

For the actual SwiftRoute implementation, similar analysis would be performed using company order data, including product, delivery zone, order quantity, and order date/time. The findings could then be used as inputs for demand forecasting and inventory planning.

8.8 Demand Forecasting:

After analyzing historical demand patterns, the next step is to estimate future demand. Demand forecasting uses historical sales or order data to predict the quantity of products that maye be required in future periods.

For SwiftRoute Logistics, demand forecasting can help the company plan inventory levels, warehouse capacity, transportation requirements, and delivery resources. Accurate demand forecasts can reduce the risk of stock-outs while also helping avoid unnecessary excess inventory.

The UCI Online Retail dataset can be used to demonstrate basic demand forecasting because it contains transaction dates and quantities. However, the actual SwiftRoute implementation would use its own historical order data, including product, quantity, Dte, time, and delivery-zone information.

8.8.1 Preparing Daily Demand Data:

The daily demand data created in Section 8.7 can be used as the starting pint forecasting.

```
daily_demand = sales_df.groupby(
   sales_df["InvoiceDate"].dt.date
)["Quantity"].sum().reset_index()

daily_demand.columns = ["Date", "Demand"]

daily_demand["Date"] = pd.to_datetime(daily_demand["Date"])

print(daily_demand.head())
```

This creates a dataset containing the date and the total quqntity sold on that date.

8.8.2 Moving Average Forecast:

A moving average provides a simple baseline forecast by calculating the average demand over a recent number of periods.

```
daily_demand["Moving_Average_7"] = (
   daily_demand["Demand"]
```

```
    .rolling(window=7)
    .mean()
)

print(daily_demand.tail())
```

Here, a 7-day moving average is used. It calculates the average demand of the previous seven days and can be used as as simple estimate of near-term demand.

A moving average is useful as a baseline because it is simple and easy to understand. More advanced forecasting methods can be compared against this baseline.

8.8.3 Visualizing Actual and Forecast Demand:

This chart allows the actual demand pattern to be compared with moving forecast. Large differences between actual demand and the moving may indicate changes in demand or periods of usually high or low demand.

```
plt.figure(figsize=(12, 5))

plt.plot(
    daily_demand["Date"],
    daily_demand["Demand"],
    label="Actual Demand"
)

plt.plot(
    daily_demand["Date"],
    daily_demand["Moving_Average_7"],
    label="7-Day Moving Average"
)

plt.title("Actual Demand and Moving Average Forecast")
plt.xlabel("Date")
plt.ylabel("Demand")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
```

```
plt.show()
```

8.8.4 Application to SwiftRoute Logistics:

For the actual SwiftRoute project, forecasting can be performed at different levels, such as:

- Product-level demand
- Daily or weekly demand
- Delivery-zone demand
- Seasonal demand
- High-demand and low-demand periods

Forecast results can support decisions such as:

- How much inventory should be maintained?
- Which products may require additional stock?
- When should replenishment be planned?
- How much warehouse capacity may be required?
- How many delivery vehicles or riders may be required during high-demand periods?

More advanced forecasting methods, including regression and time-series forecasting, can be evaluated after the initial baseline model. The forecasting method should be selected based on the quality, quantity, and characteristics of the available historical data.

Therefore, demand forecasting connects historical demand analysis with future operational planning and helps SwiftRoute make more informed inventory and logistics decisions.

8.9 Regression Analysis:

Regression analysis can used to understand the relationship between a target variable and one or more input variables. In the SwiftRoute projects, regression can be used to estimate delivery time based on operational factors such as delivery distance, order quantity, vehicle type, number of stops, delivery zone, and time of day.

The UCI Online Retail Dataset does not contain delivery-time or route-related variables, so a complete delivery-time regression model cannot be built using this dataset. Therefore, the following example

demonstrates the structure of a regression model that could be applied when SwiftRoutes's operational data becomes available.

8.9.1 Example Regression Variables:

For the actual SwiftRoute dataset:
Target variable: Delivery Time
Possible input Variables: Delivery distance, order quantity, number of stops, vehicle type, delivery zone, time of day

The model would learn relationships between these variables and delivery time. The predicted delivery time could then be used to identify deliveries that may require additional time or better scheduling.

8.9.2 Preparing Data for Regression:

For a simple numerical example, suppose SwiftRoute has a dataset containing delivery distance, order quantity, number of stops, and delivery time.

```
# Example SwiftRoute operational data

X = delivery_data[
   ["Distance", "OrderQuantity", "NumberOfStops"]
]

y = delivery_data["DeliveryTime"]

print(X.head())
print(y.head())
```

Here, X contains the input variables used by the model, while y contains the target variable that the model is expected to predict.

8.9.3 Splitting the Dataset:

The dataset can be divided into training and testing data.

```
X_train, X_test, y_train, y_test = train_test_split(
   X,
   y,
   test_size=0.2,
```

```
    random_state=42
)
```

The training data is used to learn the relationships between the variables, while the testing data is used to evaluate how well the model performs on previously unseen data.

8.9.4 Building the Regression Model:

A Linear Regression model can be created using scikit-learning.

```
model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted Delivery Times:")
print(y_pred[:10])
```

The model uses the training data to learn the relationship between delivery distance, order quantity, number of stops, and delivery time. It the uses these learned relationships to predicts delivery time for the test data.

8.9.5 Evaluating the Model:

The predicted values should be compared with the actual delivery times to understand model performance.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Mean Absolute Error:", mae)
print("Root Mean Squared Error:", rmse)
```

Mean Absolute Error (MSE) measures the average absolute difference between the actual and predicted delivery times. A time lower MAE indicates that predictions are, on average, closer to actual values.

Root Mean Squared Error (RMSE) gives importance to larger prediction errors. It can therefore help identify whether the model makes some particularly large prediction errors.

8.9.6 Business Application:

A delivery-time regression model could help SwiftRoute:

- Estimate expected delivery time for new orders.
- Identify deliveries that may take longer than expected.
- Improve delivery scheduling.
- Allocate vehicles and delivery resources more effectively.
- Identify operational factors associated with longer delivery times.
- Support better customer delivery-time estimates.

For example, if the model shows that longer delivery distances and a high number of delivery stops are associated with longer delivery times, SwiftRoute could review route planning and stop allocation.

The regression results should be interpreted together with EDA, business knowledge, and other operational information. A relationship identified by regression does not automatically prove that one variable causes another.

Since the UCI Retail Online Dataset does not contain delivery-time, distance, vehicle, or route variables, the complete regression analysis will require SwiftRoutes's operational delivery dataset.

8.10 Clustering Analysis:

Clustering is an unsupervised machine learning technique used to group similar observations based on their characteristics. For SwiftRoute Logistics, clustering can be used to to group customers or delivery zones based on factors such as order volume, order frequency, average order quantity, delivery distance, and delivery cost.

The purpose of clustering is to identify groups with similar logistics patterns so that SwiftRoute can develop different operational strategies for different customer or delivery-zone groups.

The UCI Online Retail Dataset contains customer and transaction information, so can it be used to demonstrate customer clustering. However, it does not contain delivery distance, delivery cost, or delivery-zone information. Therefore, those variables would be included when SwiftRoute's actual operational data becomes available.

8.10.1 Preparing Customer-Level Data:

The customer_summary dataset created during feature engineering contains customer-level information such as total quantity, total sales, and number of orders.

```
# Select variables for customer clustering

cluster_data = customer_summary[
   [
      "Total_Quantity",
      "Total_Sales",
      "Number_of_Orders"
   ]
].dropna()

print(cluster_data.head())
```

These variables represent different aspects of customer purchasing behaviour aand can be used to identify groups of customers with similar characteristics.

8.10.2 Standardizing the Variables:

The variables may have different scales. For example, total sales may have much larger numerical values than the number of orders. Standardization helps prevent variables with larger numerical values from having an excessive influence on the clustering process.

```
scaler = StandardScaler()
```

```
cluster_scaled = scaler.fit_transform(cluster_data)

print(cluster_scaled[:5])
```

The StandarScaler converts the variables to a common scale before clustering.

8.10.3 Applying K-Means Clustering:

K-Means is a clustering algorithm that divides observations into a specified number of groups called clusters.

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

cluster_data["Cluster"] = kmeans.fit_predict(
    cluster_scaled
)

print(cluster_data.head())
```

In this example, three clusters are created. The number of clusters should be selected based on the characteristics of the data and an appropriate evaluation method rather than being chosen arbitrarily in a final implementation.

8.10.4 Examining the Clusters:

After assigning each customer to a cluster, the average characteristics of each group can be examined.

```
cluster_summary = cluster_data.groupby(
    "Cluster"
).mean()
```

```
print(cluster_summary)
```

This summary helps compare the groups based on total quantity, total sale, and number of orders.

For example, one cluster may contain customers with relatively high order volumes and sales, while another may contain customers with lower order activity. The actual characteristics should be determined from the calculated results rather than assumed in advance.

8.10.5 Visualizing the Clusters:

A scatter plot can be used ti visualize two of the clustering variables.

```
plt.figure(figsize=(8, 5))

plt.scatter(
    cluster_data["Total_Quantity"],
    cluster_data["Total_Sales"],
    c=cluster_data["Cluster"],
    alpha=0.6
)

plt.title("Customer Clusters")
plt.xlabel("Total Quantity")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

The chart provides a visual representation of the customer groups based on quantity and sales.

8.10.6 Application to SwiftRoute Logistics:

For the actual SwiftRoute project, clustering can be extended beyond customer purchasing behaviour. Customers or delivery zones could be grouped using variables such as:

- Order volume
- Order frequency

- Average order quantity
- Average delivery distance
- Delivery cost
- Delivery frequency
- Demand level

The resulting clusters could help SwiftRoute identify different operational patterns.

For example:

- High-demand zones may require greater delivery capacity.
- Frequent-order zones may require more regular delivery scheduling.
- Long-distance zones may require route planning focused on reducing travel distance.
- Lower-demand zones may require different delivery schedules or resource allocation.

8.11 Route Optimization Approach:

Route optimization aims to find efficient delivery routes while considering delivery locations, vehicle capacity, distance, number of stops, and delivery requirements. It can help SwiftRoute reduce travel distance and transportation costs while improving vehicle utilization and delivery performance.

The UCI Online Retail Dataset does not contain, vehicle, distance, or delivery-location data. Therefore, the following is only a basic demonstration using sample SwiftRoute data.

8.11.1 Example Route Data:

```
route_data = pd.DataFrame({
   "OrderID": [101, 102, 103, 104, 105],
   "Zone": ["A", "A", "B", "B", "A"],
   "Distance": [5.2, 7.1, 4.5, 8.2, 6.0],
   "VehicleCapacity": [20, 20, 15, 15, 20],
   "OrderQuantity": [5, 4, 3, 6, 2]
})
```

```
print(route_data)
```

This data can be used to analyze delivery distance and vehicle capacity.

8.11.2 Vehicle Utilization:

The following python code helps to identify whether vehicle capacity is being efficiently utilized.

```
vehicle_load = route_data.groupby(
    "VehicleCapacity"
)["OrderQuantity"].sum().reset_index()

vehicle_load["Utilization"] = (
    vehicle_load["OrderQuantity"] /
    vehicle_load["VehicleCapacity"]
) * 100

print(vehicle_load)
```

8.11.3 Route Distance by Zone:

The delivery zones is identified with higher travel distance with the following code:

```
route_distance = route_data.groupby(
    "Zone"
)["Distance"].sum().reset_index()

print(route_distance)
```

8.11.4 Comparing Routes:

Route performance can be compared using distance, delivery time, transportation cost, vehicle, and on-time delivery rate.

```
existing_distance = 150
optimized_distance = 120

reduction = (
```

```
    (existing_distance - optimized_distance)
    / existing_distance
) * 100

print("Distance Reduction (%):", reduction)
```

8.11.5 Advanced Optimization:

For the actual SwiftRoute system, Vehicle Routing Problem (VRP) Techniques can be used to determine efficient delivery sequences while considering vehicle capacity, locations, available vehicles. delivery requirements, distance, and cost.

8.11.6 Business Application:

Route optimization can help SwiftRoute:

- Reduce travel distance and transportation costs.
- Improve vehicle utilization.
- Reduce delivery time.
- Improve on-time delivery.
- Allocate delivery resources more efficiently.

The results should be evaluated using the project's selected logistics KPIs.

8.12 KPI Calculation and Evaluation:

KPIs will be calculated to measure SwiftRoute's logistics performance and evaluate the impact of the proposed improvements.

The main KPIs are:

- **On-Time Delivery Rate:** Percentage of orders delivered on time.
- **Stock-Out Rate:** Percentage of times products are unavailable when required.
- **Average Delivery Time:** Average time taken to deliver an order.
- **Delivery Cost per Order:** Average transportation cost per order.
- **Vehicle Utilization:** Percentage of available vehicle capacity being used.
- **Inventory Turnover:** Measures how quickly inventory is sold or used.
- **Order Cycle Time:** Total time from receiving an order to final delivery.

A simple example of KPI calculation is:

```
on_time_rate = (
```

```
    delivery_data["OnTime"].mean()
) * 100

average_delivery_time = (
    delivery_data["DeliveryTime"].mean()
)

print("On-Time Delivery Rate:", on_time_rate)
print("Average Delivery Time:", average_delivery_time)
```

The KPI results can be compared across time periods, products, delivery zones, and routes. They can also be compared before and after improvements to determine whether the proposed solutions improve logistics performance.

These KPIs will provide measurable evidence for business recommendations and help SwiftRoute make better operational decisions.

8.13 Business Recommendations:

Based on the data analysis, forecasting, clustering, route optimization, and KPI evaluation, SwiftRoute can make the following recommendations:

- **Inventory:** Maintain higher stock levels for high-demand products and review slow-moving products.
- **Demand Planning:** Use demand trends and forecasts to plan inventory, warehouse capacity, and delivery resources.
- **Delivery:** Identify zones and routes with frequent delays or longer delivery times and improve scheduling.
- **Route Optimization:** Use efficient routes to reduce travel distance, transportation costs, and delivery time.
- **Vehicle Utilization:** Improve the use of available vehicle capacity through better order and route planning.
- **Customer/Zone Segmentation:** Use clusters to develop different strategies for high-, medium-, and low-demand areas.
- **KPI Monitoring:** Regularly track logistics KPIs to measure performance and identify areas for improvement.

These recommendations can help SwiftRoute improve operational efficiency, cost control, delivery performance, inventory planning, and resource allocation.

8.14 Expected Outcomes and Business Impact:

The proposed analysis is expected to help SwifRoute:

- Improve inventory planning and reduce stock-out risks.
- Improve delivery performance and reduce delays.
- Reduce unnecessary travel and transportation costs.
- Improve vehicle utilization and resource allocation.
- Support better demand forecasting and operational planning.
- Provide data-driven information for business decisions.

Overall, the project can help SwiftRoute improve operational efficiency, cost control, inventory management, and customer delivery performance.

8.15 Conclusion:

This project provides a data driven approach to improving SwiftRoute Logistics' inventory management and last-mile delivery operations. The proposed roadmap covers data collection, data quality assessment, cleaning, feature engineering, EDA, demand forecasting, regression, clustering, route optimization, and KPI evaluation.

The analysis can help SwiftRoute identify operational problems, improve inventory and delivery planning, reduce logistics costs, and make better use of available resources.

Overall, the project demonstrates how **data analysis and data science** techniques can support **better logistics planning and business decision-making.**